Polymarket Arbitrage Bot

Complete Guide

Auto-start | Position Recycler | Failover | Dashboard

Samuel Eskenasy

Creator & Owner

How the Bot Works

The bot automatically finds and exploits price differences (arbitrage) between related markets on Polymarket. It runs in a continuous cycle:

```
1. SCAN    -> Find related market pairs (e.g., Lakers vs Clippers)
2. ANALYZE -> LLM detects logical dependencies between markets
3. CALCULATE -> Math engine finds mispriced opportunities
4. BUY     -> Place orders on underpriced positions
5. RECYCLE -> Auto-sell positions to lock gains & free cash
6. REPEAT  -> Cash goes back into new opportunities
```

Architecture

Component	File	Purpose
Bot Controller	`bot_controller.py`	Main brain - manages lifecycle, trading
Polymarket Adapter	`adapters/polymarket.py`	API communication with Polymarket (REST
Dependency Detector	`engine/dependency.py`	LLM-powered detection of related markets
Math Engine	`engine/math.py`	Calculates arbitrage opportunities and f
Execution Engine	`engine/execution.py`	Places and manages orders
Dashboard	`dashboard.py`	Web UI for monitoring and control
Config	`config.py`	Loads settings from `.env`

Key Concepts

Wallet Balance vs Positions vs Net Equity

Metric	Meaning
Wallet Balance	Cash (USDC) sitting in your Polymarket t
Reserved in Live Orders	Cash locked in open orders waiting to be
Position Value (Pos MTM)	Current market value of shares you own
Unrealized PNL	Paper profit/loss on open positions
Realized PNL	Actual profit/loss from closed positions
Net Equity	Wallet + Position Value = total worth

How Money Flows

```
Cash (Wallet) --[bot buys shares]--> Positions
                                        |
                                  Market price changes
                                        |
                                    Unrealized PNL (±)
                                        |
                        Position sold or market resolves
                                        |
                                    Realized PNL -> Cash back
```

What Happens When Wallet Reaches $0

- Bot has a safety guard: stops placing new orders when balance < $2.00
- Position recycler kicks in at < $15 (emergency sell)
- Existing positions and open orders are NOT affected
- Cash returns when: orders fill, positions are sold, or markets resolve

Configuration (.env)

Core Settings

```
EXECUTION_MODE=LIVE          # LIVE or PAPER (paper = simulation only)
MIN_PROFIT_THRESHOLD=0.001   # Minimum profit per trade ($0.001)
MAX_POSITION_SIZE=5.0        # Max dollars per order
MAX_DAYS_TO_RESOLUTION=120   # Only trade markets resolving within 120 days
```

Auto-Start

```
AUTO_START=1                 # Bot starts trading when Docker starts
```

When Mac lid opens -> Docker Desktop starts -> Container starts -> Bot starts trading (3s delay). No manual clicks needed.

Position Recycler (Automatic Buy/Sell Cycle)

```
RECYCLE_ENABLED=1            # Enable automatic position recycling
RECYCLE_INTERVAL_HOURS=4     # Run recycler every 4 hours
RECYCLE_MIN_HOLD_HOURS=2     # Don't sell positions younger than 2 hours
RECYCLE_MAX_HOLD_HOURS=12    # Force sell positions older than 12 hours
RECYCLE_MIN_CASH=15.0        # Emergency sell if cash drops below $15
```

Recycler Logic:

1. Cash < $15 -> emergency sell the most profitable position
2. Position held > 12 hours -> force sell (free up capital)
3. Position profitable + held > 2 hours -> sell to lock in gains

Failover (Mac + PC Setup)

Mac (primary):

```
BOT_ROLE=primary
```

Windows PC (standby):

```
BOT_ROLE=standby
PRIMARY_HEALTH_URL=http://<mac-ip>:8080/health
```

Mac State	What Happens
Mac OPEN	Mac trades. PC watches, does NOT trade
Mac CLOSES	PC detects Mac is down (90s). PC takes o
Mac OPENS again	PC detects Mac is back. PC stops. Mac re

Safety Limits

```
# These are in bot_controller.py constants:
MAX_LIVE_ORDERS = 8                    # Max simultaneous open orders
STALE_ORDER_TIMEOUT_SECONDS = 120      # Auto-cancel unfilled orders after 2 min
MIN_BALANCE_RESERVE = 2.0              # Stop trading if wallet < $2
```

Dashboard

Access

- URL: http://localhost:8080 (or your domain)
- Login: admin / polyarb2026 (configurable in .env)

API Endpoints

Endpoint	Method	Auth	Purpose
`/`	GET	Yes	Dashboard UI
`/health`	GET	No	Health check (used by failover)
`/api/state`	GET	Yes	Full bot state
`/api/start`	POST	Yes	Start the bot
`/api/stop`	POST	Yes	Stop the bot
`/api/settings`	POST	Yes	Update settings
`/api/positions`	GET	Yes	List all positions
`/api/close_position`	POST	Yes	Sell a specific position

Close a Position via API

```
curl -u admin:polyarb2026 -X POST http://localhost:8080/api/close_position \
  -H "Content-Type: application/json" \
  -d '{"token_id": "FULL_TOKEN_ID_HERE"}'
```

List Positions

```
curl -u admin:polyarb2026 http://localhost:8080/api/positions | python3 -m json.tool
```

Docker Commands

Start the bot

```
cd ~/poly_arb_bot
docker compose up -d --build
```

Stop the bot

```
docker compose down
```

View logs

```
docker logs poly_arb_bot -f              # Live follow
docker logs poly_arb_bot --tail 50       # Last 50 lines
docker logs poly_arb_bot 2>&1 | grep "TRADE SIGNAL"   # Only trade signals
docker logs poly_arb_bot 2>&1 | grep "RECYCLER"       # Only recycler activity
```

Check wallet balance

```
docker exec poly_arb_bot python3 -c "
from poly_arb_bot.adapters.polymarket import PolymarketAdapter
a = PolymarketAdapter()
bal = a.get_balance_allowance()
print(f'Wallet: \${int(bal[\"balance\"]) / 1e6:.2f}')
"
```

Cancel all open orders

```
docker exec poly_arb_bot python3 -c "
from poly_arb_bot.adapters.polymarket import PolymarketAdapter
a = PolymarketAdapter()
for o in a.get_open_orders():
    a.cancel_order(o['id'])
    print(f'Cancelled {o[\"side\"]} ...{o[\"asset_id\"][-8:]}')
"
```

Check actual shares on exchange

```
docker exec poly_arb_bot python3 -c "
from poly_arb_bot.adapters.polymarket import PolymarketAdapter
from py_clob_client.clob_types import BalanceAllowanceParams, AssetType
a = PolymarketAdapter()
TOKEN_ID = 'PASTE_TOKEN_ID_HERE'
params = BalanceAllowanceParams(asset_type=AssetType.CONDITIONAL, token_id=TOKEN_ID, signature_type=2)
bal = a.client.get_balance_allowance(params)
print(f'Shares: {int(bal[\"balance\"]) / 1e6:.2f}')
"
```

Best Trading Times

Time (Israel)	US Eastern	Activity
4pm - 7pm	9am - 12pm	**Highest** - US markets open, news drop
7pm - 12am	12pm - 5pm	**High** - Active US trading
12am - 4am	5pm - 9pm	**Medium** - Evening news
4am - 4pm	9pm - 9am	**Lower** - Fewer traders, wider spreads

Peak days: Monday-Tuesday (weekend news catch-up)

Troubleshooting

Bot stuck at "Live order limit reached (8/8)"

Old unfilled orders are blocking new trades. Cancel them:

```
# Orders auto-cancel after 2 minutes, but for immediate fix:
docker exec poly_arb_bot python3 -c "
from poly_arb_bot.adapters.polymarket import PolymarketAdapter
a = PolymarketAdapter()
for o in a.get_open_orders(): a.cancel_order(o['id'])
print('All orders cancelled')
"
```

"Wallet balance too low" warning

The recycler should handle this automatically. If it doesn't, manually sell a position from Polymarket.com or via the API.

"not enough balance / allowance" on SELL

The position book may overstate shares. The bot's close_position function now checks actual exchange balance before selling.

Geographic restriction (403)

The bot must run from an IP that Polymarket accepts (residential IPs). Run from your Mac or PC at home. VPS/cloud IPs are blocked.

WebSocket disconnection

The bot auto-reconnects. If persistent, restart: docker compose restart

File Structure

```
poly_arb_bot/
??? .env                    # Configuration (API keys, settings)
??? docker-compose.yml      # Docker orchestration
??? Dockerfile              # Container build
??? dashboard.py            # Flask web dashboard
??? bot_controller.py       # Main bot logic + recycler
??? config.py               # Settings loader
??? adapters/
?   ??? polymarket.py       # Polymarket API adapter
??? engine/
?   ??? math.py             # Arbitrage math
?   ??? execution.py        # Order execution
?   ??? dependency.py       # LLM dependency detection
??? templates/
?   ??? dashboard.html      # Dashboard UI
??? data/
?   ??? runtime_state.json  # Persisted positions & state
??? scripts/
    ??? generate_api_keys.py
    ??? test_order.py
```

Important Notes

1. Never run two bots trading simultaneously on the same Polymarket account - they will conflict
2. The failover system ensures only one bot trades at a time (Mac = primary, PC = standby)
3. Position book may drift from actual exchange balance - the recycler checks real balances before selling
4. You can always sell directly on Polymarket.com - click the Sell button on any position
5. Docker Desktop must be set to start on login for auto-start to work when Mac wakes up